

Hedgingbot — Deep Architecture & Risk Review

Read-only audit performed with Ox Alpha Free through OpenCode

Executive Summary

The audit found that the Hedgingbot's mathematical core is genuinely sound and unusually well-tested. The main risks are concentrated at the boundary between the pure mathematical/decision layers and the live operating environment: unwired safety guards, shared-state synchronization, beta freshness, static account assumptions, and MT5 data handling.

Current status: Phase 1 is measurement-only and the MT5 connector contains no order-placement path. Therefore, the identified weaknesses do not currently create trading P&L; exposure. They should nevertheless be resolved before Phase 2 introduces live order execution.

1. Factor-Exposure Mathematics

Verdict: Sound, with documented modelling limits. The audit found that signed notional, beta aggregation, equity normalization, and the factor-leverage convention are internally consistent. Portfolio variance treatment is valid under the orthogonality assumption.

Important caveats:

- Residual/idiosyncratic risks may be correlated, making some portfolio-risk statistics optimistic.
- Log-return betas are applied linearly and therefore do not capture convexity during very large moves.
- Weekly betas are used for daily decisions; the horizon-invariance assumption can fail during regime changes.
- The unmapped-symbol volatility placeholder is arbitrary, but unmapped symbols are hard-blocked from hedging.

2. Gram-Schmidt Factor Construction

Verdict: Correctly implemented. The sequential residualization is mathematically valid because previously constructed factors are mutually orthogonal. Rescaling preserves correlations and therefore orthogonality. Degenerate-factor, sample-size, non-finite, and ragged-input guards are present. The order dependence of the factor construction is documented and tested.

3. Factor-Leverage Consistency

Verdict: Consistent. `ExposureReport.factor_leverage` is the central convention used by breach detection, reporting, and hedge simulation. Weekly-to-daily volatility scaling is performed once during persistence and is covered by tests.

Gap: Factors missing from the caps configuration are silently treated as unlimited. A configuration typo could therefore remove a risk limit without an explicit failure.

4. Hedge Decision Logic — Unintended Trade Paths

- **Collateral check blind spot:** pre-existing breaches can be worsened because the new-breach check focuses on factors that were inside their cap before the proposed hedge.
- **No hedge provenance:** existing hedges are judged using the instrument's current dominant factor rather than a recorded reason/breach that caused the hedge to be opened.
- **Minimum-lot early return:** failure to fit one candidate can stop the search before another viable candidate is considered.
- **Partial hedge repetition:** gross-ceiling limits can leave a breach standing, causing another hedge decision on the next cycle.

5. Anti-Churn Protections

The band geometry is logically sound and the static convergence test passes. However, two significant weaknesses were identified:

Guard 6 is not wired into the live loop. The daily hedge counter exists, but the live measurement loop does not pass the current hedge count into `hedge_decide()`, and `note_hedge()` is not called. The daily action budget is therefore currently cosmetic.

Equity-denominator oscillation. Because leverage is exposure divided by equity, a change in equity alone can move leverage through the open and unwind bands. This can cause unnecessary hedge/open-close cycles during volatile periods.

6. Collateral-Damage Checks

- Pre-existing breaches are not adequately protected from being worsened.
- Uncapped factors are excluded from collateral checks.
- The system checks cap compliance but does not assert that total portfolio daily risk actually decreases after a hedge.

7. Numerical, Data, State & Concurrency Risks

Risk	Location	Why it matters
N1 — Beta staleness	<code>store.py</code> / <code>measure_exposure.py</code>	Betas are checked when loaded, not continuously during a long-running loop.
N2 — Shared-state clobber	<code>shared.py</code> / <code>measure_exposure.py</code>	A stale writer may overwrite a newer halt flag or high-water mark.
N3 — No lock implementation	<code>exposure.py</code> / <code>src</code>	Documented lock configuration exists, but no actual locking implementation was found.
N4 — Bad tick silently dropped	<code>mt5_reader.py</code>	A bad price can make a live position disappear from the risk book.
N5 — Tick-size fallback	<code>mt5_reader.py</code>	Fallback from tick size to point can silently distort notional and leverage.
N6 — Uncaught spec error	<code>mt5_reader.py</code> / <code>model.py</code> / <code>measure_exposure.py</code>	Unexpected broker data can terminate the observe loop.
N7 — Yahoo cache data hydration	<code>data_hydrating.py</code>	UTC day-label differences can create alignment risk if downstream gates are bypassed.
N8 — Partial-overlap stipulate	<code>stipulate.py</code>	Different instruments can receive betas from different historical regimes.

8. MT5 Live-Safety Review

- The strongest safety gate is that Phase 1 genuinely contains no order-placement path.
- The avoid-symbol list is derived from static configuration rather than the complete live account book.
- SILVER has a conflicting dual role: it appears as both a trader symbol and a hedge candidate.
- Hedge identification relies on the configured magic number, with no runtime validation that it differs from the trader's magic number.
- Netting vs hedging account detection is reported but not structurally enforced.
- Broker data such as tick value is surfaced by preflight checks but is not continuously revalidated inside the loop.

9. Ten Highest-Risk Weaknesses

- 1. Daily-budget guard unwired** — `hedges_today` is not passed through the live loop and `note_hedge()` is never called.
- 2. Collateral check blind spot** — Existing breaches and uncapped factors can be worsened by a hedge.
- 3. Shared kill-switch clobber** — A stale full-state save can potentially resurrect `halted=False` or regress the high-water mark.
- 4. One-time beta staleness check** — A long-running process can continue using arbitrarily old betas.
- 5. Static avoid list / SILVER conflict** — Live manual or other-EA positions may not be protected; SILVER has conflicting roles.
- 6. Silent bad-tick position drop** — A live position can disappear from the measured risk book without a warning.

- 7. No lock implementation** — Concurrent overlay instances could produce duplicate actions once trading is enabled.
- 8. Tick-size fallback** — Incorrect contract scaling can propagate into all downstream leverage calculations.
- 9. Breach starvation** — An unhedgeable worst breach can prevent a smaller, hedgeable breach from being addressed.
- 10. Unwind provenance gap** — Changing beta rankings can cause existing hedges to be evaluated against the wrong factor.

10. Overall Assessment

Mathematical engine: Strong.

Factor model: Strong.

Unit consistency: Strong.

Testing: Strong.

Decision engine: Needs hardening.

State management: Needs fixing.

Live safety: Not ready.

Order execution: Do not enable yet.

The audit's central conclusion is that the project is in a good position precisely because these weaknesses were found before Phase 2. The recommended engineering approach is to address the safety and state-management weaknesses first, then validate the decision layer again before adding order execution.

Source Note

This report is based on the read-only architecture and risk review output supplied from the Ox Alpha Free audit of the Hedgingbot repository. No code changes were made as part of that audit.